

Computer DLOC Lab-I

CE509 Agentic AI

Lab Report

MedAgent-CDSS: A Multi-Agent Clinical Decision Support System for Educational Simulation

(Educational simulation using synthetic patient data only)

Item	Details
Problem Statement Title	Clinical decision support multi-agent system
Domain	Healthcare and Life Sciences
Problem Statement Number & Title	PS No. – Clinical decision support multi-agent system
Student Name	Aamir Khan
Roll Number	Roll Number
Email	aamirak.pathaan@gmail.com
Course Name	M.Tech Computer Engineering – CE509 Agentic AI (Computer DLOC Lab-I)
Semester	Semester
Submission Date	DD Month 2026

1. Objective

The objective of this work is to design and implement MedAgent-CDSS, a multi-agent system that analyses a synthetic patient case – symptoms, medical history and laboratory results – and produces an educational report listing conditions to consider and diagnostic steps that could be considered next. Each sub-task is handled by a specialised agent, and the agents are orchestrated with LangGraph through a shared, typed state. The reasoning agents use a local large language model (Gemma 3 4B via Ollama) with structured outputs, grounded by retrieval-augmented generation (RAG) over a small educational knowledge base. A deterministic safety layer ensures the output never claims certainty, never recommends treatment and always carries an educational disclaimer.

2. Work Done

2.1 Problem Understanding

- Clinical decision support combines several distinct steps: checking the data, interpreting symptoms, reviewing history, comparing labs with reference ranges, recalling relevant knowledge, forming a differential diagnosis and planning investigations [1].
- A single large prompt mixes all of these steps, is hard to test and can hallucinate. Splitting the work into agents with narrow roles and structured outputs makes every step inspectable and testable.
- Because this is a teaching tool, the system must never present a diagnosis as certain, must never suggest treatment, and must use **synthetic data only**.

2.2 Task Decomposition

- **Data handling (deterministic):** validate and structure the case; analyse symptoms, history and labs.
- **Knowledge (deterministic):** build a query from the three analyses and retrieve educational documents (RAG) [2].
- **Reasoning (LLM with rule-based fallback):** generate conditions to consider; propose diagnostic steps.
- **Control (deterministic):** guardrails on the differential, safety review with a feedback loop, report generation.
- Design rule: the LLM is used only where language reasoning adds value; data handling, ranking constraints and safety stay in code.

2.3 Agent Roles and Responsibilities

Table 1: Agents, their inputs (read from the shared state) and outputs (written to it)

Agent	Responsibility	Reads	Writes	Type
Patient Data Agent	Validate input with Pydantic; report missing/invalid fields; stop the workflow if invalid	patient_input	patient, validation	Rules
Symptom Analysis Agent	Key features, duration category, body systems, warning (red-flag) symptoms	patient	symptom_analysis	Rules
Medical History Agent	Classify conditions, medication classes (context only), allergies, risk factors	patient	history_analysis	Rules
Laboratory Analysis Agent	Compare values with reference ranges: low / normal / high / invalid / unknown	patient	lab_analysis	Rules
RAG Retriever	Build query; TF-IDF + lab-signal retrieval of top-4 documents	three analyses	retrieved_knowledge	Rules
Differential Diagnosis Agent	Up to 5 conditions to consider with evidence; guardrails applied	analyses, knowledge, safety_feedback	differential_diagnosis	LLM + rules
Diagnostic Planning Agent	Diagnostic steps that could be considered, each with a rationale	analyses, differential, safety_feedback	diagnostic_plan	LLM + rules
Safety Review Agent	Detect certainty, treatment, unsafe or necessity wording; risk level; revise or redact	differential, plan, red flags	safety_review, safety_feedback	Rules
Report Generator	Assemble the educational report with disclaimer	whole state	final_report	Rules

2.4 Communication Design

- **Shared state:** all agents communicate through one `ClinicalState` (a typed dictionary). Each agent reads only the keys it needs and writes only its own keys, so every message between agents is structured data, not free text.
- **Parallel fan-out / fan-in:** after validation, the Symptom, History and Laboratory agents run in parallel; the RAG Retriever waits for all three. Log fields use an append reducer so parallel writes merge safely.
- **Conditional routing:** invalid input goes straight to the report; the Safety Review Agent either finalises or sends feedback back.

- **Feedback loop:** if LLM-generated text breaks a safety rule, the Safety Review Agent writes `safety_feedback`, which the Diagnosis and Planning agents include in their next prompt (maximum one revision). Remaining violations are redacted.
- **Trace:** every node appends a message (agent, status, reads, writes, message, duration) that the UI shows as the agent-communication trace.

2.5 Tools Used

Table 2: *Tools and technologies*

Tool	Purpose in the project
Python 3.10	Implementation language
LangGraph 1.2	StateGraph orchestration: parallel branches, conditional edges, feedback loop [3]
LangChain (langchain-core, langchain-openai)	ChatOpenAI client and structured-output parsing [4]
Ollama + Gemma 3 4B	Local LLM served through an OpenAI-compatible API; JSON mode because Gemma 3 in Ollama has no tool calling [5], [6]
Pydantic v2	Input validation and output schemas for every agent [7]
TF-IDF retriever (pure Python)	Reproducible, offline RAG over 13 educational documents [8]
Streamlit	Web user interface [9]
pytest	101 automated tests; LLM calls mocked or replayed [10]
Docker, Git/GitHub	Containerised run; version control and sharing

2.6 Architecture and Workflow

Figure 1 shows the system architecture and Figure 2 the LangGraph workflow, including the parallel analysis stage, the invalid-input exit and the safety feedback loop.

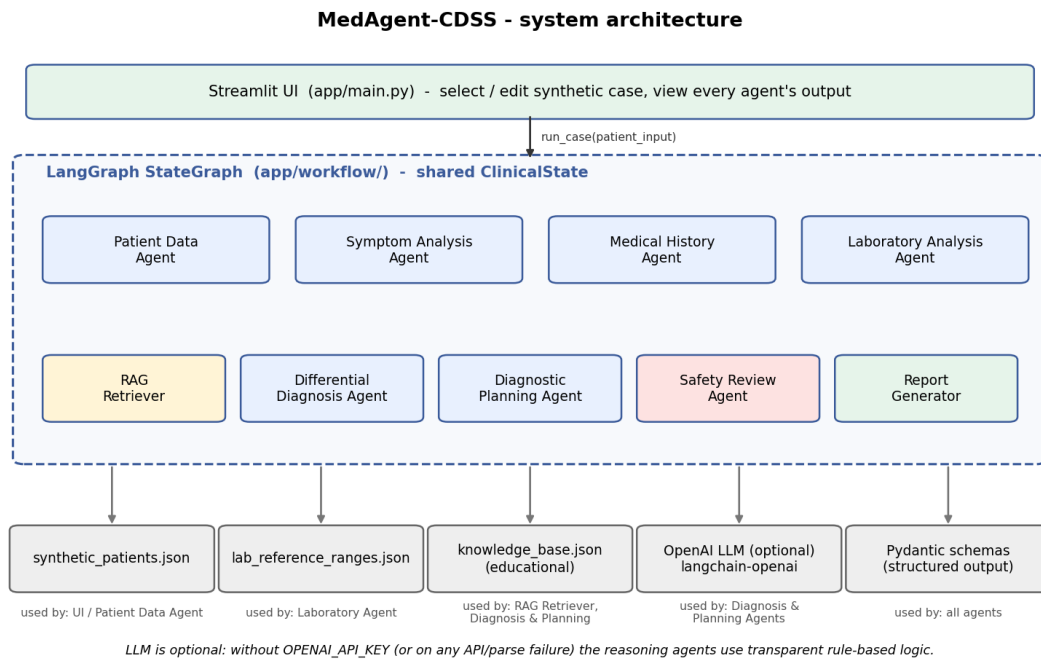


Figure 1: System architecture of MedAgent-CDSS

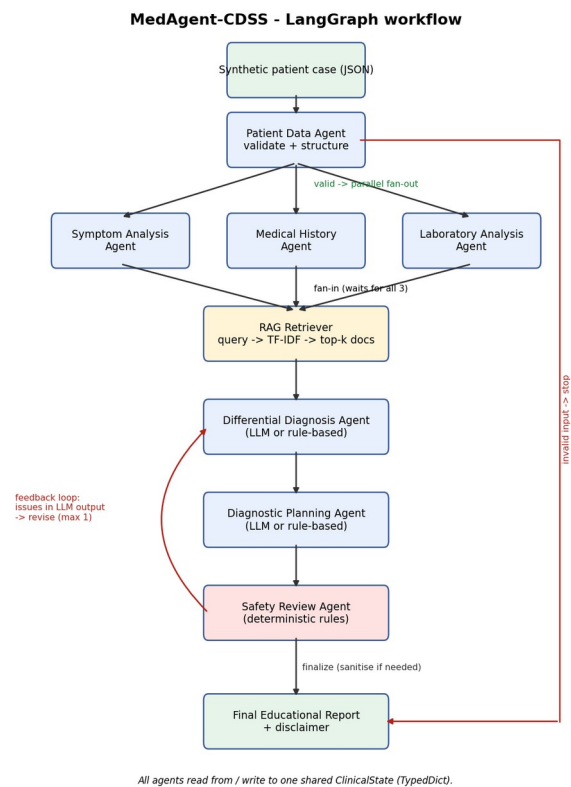


Figure 2: LangGraph workflow with parallel fan-out, conditional routing and safety feedback loop

3. Results / Output

3.1 Agent Interaction Flow

Table 3 is the actual agent-communication trace recorded for synthetic case SIM-001 (45-year-old male, fever, cough and fatigue for 5 days, WBC 13,500 cells/ μ L, CRP 48 mg/L).

Table 3: Agent-communication trace for SIM-001 (rule-based mode)

#	Agent	Message passed on through the shared state
1	Patient Data Agent	Patient record valid; structured case shared with analysis agents.
2	Symptom / History / Lab (parallel)	3 symptoms (constitutional, respiratory); 1 condition, 1 medication; 2 labs outside reference range.
3	RAG Retriever	Query “fever cough fatigue hypertension WBC high CRP high” → KB-001, KB-012, KB-005, KB-003.
4	Differential Diagnosis Agent	3 candidates via rule-based reasoning.
5	Diagnostic Planning Agent	6 diagnostic steps via rule-based reasoning.
6	Safety Review Agent	Risk low; passed.
7	Report Generator	Report status: completed.

3.2 Sample Inputs and Outputs

Table 4: Sample synthetic inputs and the system’s outputs

Input (synthetic)	Output
SIM-001: 45 M; fever, cough, fatigue, 5 days; hypertension; WBC 13,500, CRP 48	Conditions to consider: community-acquired pneumonia (higher), urinary tract infection (moderate), influenza-like illness (moderate). Steps: chest X-ray, pulse oximetry, urinalysis. Safety: low risk.
SIM-005: 61 M; chest pain, shortness of breath, sweating, 2 hours; hypertension, type 2 diabetes, smoker; glucose 142	Acute coronary syndrome (higher) listed first; known diabetes capped at lower by a guardrail. First step: prompt in-person clinician review, then ECG and troponin. Safety: high risk, flagged for human review.
SIM-007: 26 F; burning and frequent urination; WBC 11,800; CRP –4 (invalid)	Urinary tract infection (higher); CRP marked invalid and excluded; plan includes “verify or repeat the invalid laboratory entries: CRP”.
SIM-008: age 130, sex missing	Workflow stopped at validation: “age: Input should be less than or equal to 120”, “sex: Field required”.

3.3 Screenshots of the System Running

The screenshots below were captured from the running Streamlit application (rule-based mode). Additional screenshots are in the Appendix.



A Multi-Agent Clinical Decision Support System for Educational Simulation

⚠️ Educational simulation only. This system does not provide medical diagnosis, treatment, or professional medical advice. All patient cases are synthetic.

Agent outputs 10. Final educational report Agent trace

1. Patient information

```
{
  "patient_id": "SIM-001"
  "age": 45
  "sex": "Male"
  "symptoms": [
    0: "fever"
    1: "cough"
    2: "fatigue"
  ]
  "symptom_duration": "5 days"
  "medical_history": [
    0: "hypertension"
  ]
  "medications": [
    0: "antihypertensive medication"
  ]
}
```

Figure 3: Streamlit dashboard – SIM-001: disclaimer, patient information, validation, symptom and history analysis

6. Retrieved educational context (RAG)

Query: chest pain shortness of breath sweating hypertension type 2 diabetes smoker Glucose_fasting high

> KB-007 - Type 2 diabetes mellitus and hyperglycemia (score 0.4894)

> KB-010 - Acute coronary syndrome (warning presentation) (score 0.448)

> KB-001 - Community-acquired pneumonia (score 0.2208)

> KB-004 - Dengue fever (score 0.0918)

7. Differential diagnosis candidates (conditions to consider)

3 condition(s) to consider were derived by matching the case's symptoms, history and out-of-range labs against the retrieved educational knowledge. Levels are relative, not probabilities.

Guardrail: 'Type 2 diabetes mellitus and hyperglycemia' is already in the patient's history; set to lower.

Acute coronary syndrome (warning presentation) - higher consideration (cardiovascular)

Supporting: Reported symptom: chest pain; Reported symptom: shortness of breath; Reported symptom: sweating; History factor: hypertension; History factor: type 2 diabetes

Knowledge refs: KB-010

Community-acquired pneumonia - moderate consideration (respiratory infection)

Supporting: Reported symptom: chest pain; Reported symptom: shortness of breath

Figure 4: SIM-005 (red-flag case): retrieved knowledge, guardrail note and conditions to consider

3.4 Evaluation

The evaluation script (`python -m app.evaluation`) was run on all 8 synthetic cases (7 valid, 1 invalid) plus 7 extra malformed inputs and a hand-written labelled set of 18 unsafe and 12 safe sentences. Table 5 lists the measured values.

Table 5: Evaluation results (rule-based mode, measured 27 Sep 2026)

Metric	Result
Input validation accuracy	100% (15/15)
Workflow completion rate (valid cases)	100% (7/7)
Agent execution success rate	100% (65/65 agent runs)
Structured output validity (Pydantic)	100% (14/14)
Disclaimer present in report	100% (8/8)
Top candidate = case's intended teaching pattern	100% (7/7)
Safety violation detection rate	100% (18/18)
Safety false-positive rate	0% (0/12)
Latency per case (mean / max)	7.4 ms / 11.9 ms

These figures describe software behaviour on synthetic data written alongside the rules, so they show that the pipeline works as designed; they are **not** a measure of clinical accuracy.

The Gemma 3 4B model (Q4_K_M, temperature 0, JSON mode) was tested by sending the agents' actual prompts to the local Ollama server. Table 6 summarises the observed behaviour.

Table 6: Observed behaviour of Gemma 3 4B on the agents' prompts

Call	Time	Valid JSON	Observation
Diagnosis, SIM-001	19.8 s	Yes	Sensible ranking; passed the safety review with no issues.
Diagnosis, SIM-005 (first prompt)	15.2 s	Yes	Ranked known diabetes above chest-pain ACS; ungrounded 'Hypertension' and 'Dengue' candidates.
Diagnosis, SIM-005 (tightened prompt)	13.4 s	Yes	ACS first, but invented a patient location ("India") as evidence for dengue.
Planning, SIM-005	12.7 s	Yes	Good steps (ECG, troponin) but "is warranted", "should be measured"; clinician-review label placed on the ECG step.

The automated test suite (python -m pytest) contains **101 tests** and all pass. They cover every agent, retrieval, output structure, safety detection and redaction, the complete workflow on every case, the feedback loop, API failure and malformed-output fallback, the local-server path, the Streamlit UI and the recorded Gemma replies above (replayed without calling the model).

3.5 GitHub Repository

Source code, data, tests, documentation and screenshots:
https://github.com/AK-Aamir-Khan/clinical_multi-agent_cdss

4. Observations & Learning

4.1 What Worked Well

- Splitting the work into narrow agents with Pydantic outputs made each step easy to test and to explain.
- LangGraph’s shared state, parallel branches and conditional edges expressed the workflow cleanly, including the feedback loop.
- Gemma 3 4B returned valid JSON on every call in JSON mode, and the rule-based fallback kept the system working when the model was unavailable.

4.2 Challenges and Solutions

- **No tool calling in Gemma 3 (Ollama):** switched to JSON mode and added a generated JSON template to the prompt; replies are still validated with Pydantic.
- **Wrong ranking and invented evidence by the small model:** tightened the prompt, then added deterministic guardrails – a grounding check, a known-history cap and red-flag priority.
- **Prescriptive wording (“is warranted”, “should be measured”):** added necessity rules to the Safety Review Agent and routed its feedback to the Planning Agent as well.
- **Over-flagging:** “breathlessness on exertion” was first treated as a red flag; the red-flag list was narrowed after evaluation.

4.3 Key Learning Points

- A small local LLM is useful for structure and explanation, but ranking, grounding and safety must be enforced in code.
- Prompt engineering alone reduced errors but did not remove hallucinated evidence.
- Recording real model outputs as test fixtures turns model behaviour into repeatable regression tests.

5. Conclusion

MedAgent-CDSS demonstrates a working multi-agent clinical decision support simulation built with LangGraph. Nine components cooperate through a shared, typed state with parallel execution, conditional routing and a safety feedback loop. A local Gemma 3 4B model provides structured reasoning, grounded by RAG and constrained by deterministic guardrails and a rule-based safety agent. The system runs offline, is covered by 101 passing tests and produces clearly labelled educational reports. It is an educational simulation only and is not intended for real clinical use.

6. References

- [1] R. T. Sutton, D. Pincock, D. C. Baumgart, D. C. Sadowski, R. N. Fedorak and K. I. Kroeker, “An overview of clinical decision support systems: benefits, risks, and strategies for success,” *npj Digital Medicine*, vol. 3, art. 17, 2020.
- [2] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [3] LangChain, “LangGraph documentation.” [Online]. Available: <https://langchain-ai.github.io/langgraph/> (accessed Sep. 2026).
- [4] LangChain, “LangChain Python documentation – structured output.” [Online]. Available: <https://python.langchain.com/> (accessed Sep. 2026).
- [5] Gemma Team, Google DeepMind, “Gemma 3 technical report,” *arXiv:2503.19786*, 2025.
- [6] Ollama, “OpenAI compatibility.” [Online]. Available: <https://github.com/ollama/ollama/blob/main/docs/openai.md> (accessed Sep. 2026).
- [7] Pydantic, “Pydantic documentation.” [Online]. Available: <https://docs.pydantic.dev/> (accessed Sep. 2026).
- [8] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [9] Streamlit, “Streamlit documentation.” [Online]. Available: <https://docs.streamlit.io/> (accessed Sep. 2026).
- [10] pytest, “pytest documentation.” [Online]. Available: <https://docs.pytest.org/> (accessed Sep. 2026).

Appendix

A. Important Prompts

Every LLM prompt follows the structure **ROLE – TASK – INPUT – CONSTRAINTS – OUTPUT FORMAT**. The shared constraints and the Differential Diagnosis system prompt (from `app/prompts.py`):

A.1 Shared constraints

```
SHARED_CONSTRAINTS = """CONSTRAINTS:
- This is an EDUCATIONAL SIMULATION using SYNTHETIC patient data. It is not real clinical care.
- Never state or imply certainty (no "definitely", "confirmed", "the patient has").
- Never recommend, name doses of, start, stop or change any medication or treatment.
- Use ONLY the supplied case context and the supplied educational knowledge; do not invent test results.
- If information is insufficient, say so explicitly.
- Keep language neutral and educational."""
```

A.2 Differential Diagnosis Agent – system prompt

```
DIAGNOSIS_SYSTEM = f"""ROLE: You are the Differential Diagnosis Agent in a multi-agent clinical decision support SIMULATION used for teaching.
```

```
TASK: From the structured outputs of the Symptom, Medical History and Laboratory agents, and the retrieved educational knowledge, list up to 5 conditions or condition categories a student should CONSIDER, with the evidence for and against each.
```

```
{SHARED_CONSTRAINTS}
```

```
- consideration_level is a relative label ("higher", "moderate", "lower"), not a probability.
- knowledge_refs must only contain IDs from the RETRIEVED KNOWLEDGE section; every candidate needs at least one.
- Candidates must help explain the CURRENT symptoms. Conditions already listed in history_conditions are context (risk factors), not new candidates.
- If red_flags are present, list first the condition that best explains those warning symptoms.
- Only cite evidence that appears in the case context or in the retrieved knowledge text.
```

```
OUTPUT FORMAT: Return the DifferentialDiagnosisOutput structure."""
```

A.3 Diagnostic Planning Agent – system prompt

```
PLANNING_SYSTEM = f"""ROLE: You are the Diagnostic Planning Agent in a multi-agent clinical decision support SIMULATION used for teaching.
```

```
TASK: For the candidate conditions produced by the Differential Diagnosis Agent, suggest up to 6 diagnostic steps (tests or assessments) that could be CONSIDERED next, and explain why each might help distinguish between the candidates.
```

```
{SHARED_CONSTRAINTS}
```

```
- Suggest diagnostic steps only - no medication, therapy or procedure intended as treatment.
```

- Do not claim any step is mandatory or medically required; use wording such as "could be considered".
- If red-flag symptoms are present, the first step should be prompt clinician review.

OUTPUT FORMAT: Return the DiagnosticPlanOutput structure."""

In JSON mode the system prompt is extended with “RESPONSE FORMAT: Reply with ONE JSON object only ...” followed by a template generated from the Pydantic schema, for example "consideration_level": "higher | moderate | lower".

B. Selected Code Snippets

B.1 LangGraph workflow construction (app/workflow/graph.py)

```
def build_graph(llm=None, settings: Optional[Settings] = None):
    settings = settings or get_settings()
    graph = StateGraph(ClinicalState)
    for name, fn in {**build_analysis_nodes(), **build_reasoning_nodes(llm,
settings)}.items():
        graph.add_node(name, fn)

    graph.add_edge(START, "patient_data")
    graph.add_conditional_edges("patient_data", route_after_validation, ANALYSIS_NODES
+ ["final_report"])
    graph.add_edge(ANALYSIS_NODES, "knowledge_retrieval") # fan-in: waits for all
three
    graph.add_edge("knowledge_retrieval", "differential_diagnosis")
    graph.add_edge("differential_diagnosis", "diagnostic_planning")
    graph.add_edge("diagnostic_planning", "safety_review")
    graph.add_conditional_edges("safety_review", route_after_safety,
["differential_diagnosis", "final_report"])
    graph.add_edge("final_report", END)
    return graph.compile()
```

B.2 Structured LLM call with fallback (app/llm.py)

```
def invoke_structured(llm, schema: Type[T], messages: List[Tuple[str, str]],
method: Optional[str] = None) -> Tuple[Optional[T],
Optional[str]]:
    """Call the LLM and parse its answer into `schema`.

    Returns (result, error). Any failure - API error, timeout, malformed or
    schema-invalid output - is returned as a sanitised error string so the
    calling agent can fall back to its deterministic logic.
    """
    if llm is None:
        return None, None
    method = method or get_settings().structured_output_method
    try:
        if method == "json_mode": # the model is not told the schema, so describe it
            messages = _with_json_instructions(messages, schema)
            structured = llm.with_structured_output(schema, method=method)
            result = structured.invoke(messages)
            if isinstance(result, dict): # some providers return dicts
                result = schema.model_validate(result)
            if not isinstance(result, schema):
```

```

        return None, f"LLM returned unexpected type {type(result).__name__}"
    return result, None
except Exception as exc: # noqa: BLE001
    return None, sanitize_error(exc)

```

B.3 Guardrails on LLM candidates (app/agents/diagnosis_agent.py, excerpt)

```

notes: List[str] = []
kept: List[DiagnosisCandidate] = []
for cand in output.candidates:
    docs = [d for d in (self.kb.get(ref) for ref in cand.knowledge_refs) if d]
    if from_llm:
        if not docs:
            notes.append(f"Removed '{cand.condition}': not linked to any
retrieved knowledge.")
            continue
        if not any(ev["symptom_hits"] or ev["lab_hits"]
                    for ev in (self._evidence(d, context, lab_results) for d in
docs)):
            notes.append(f"Removed '{cand.condition}': no reported symptom or
abnormal lab "
                        "matches the cited knowledge.")
            continue
        if self._is_known_condition(cand.condition,
context["history_conditions"]):
            if cand.consideration_level != "lower":
                notes.append(f"'{cand.condition}' is already in the patient's
history; set to lower.")
                cand.consideration_level = "lower"
                note = "Already documented in the history - context rather than a new
explanation."
                if note not in cand.missing_or_conflicting_evidence:
                    cand.missing_or_conflicting_evidence.append(note)
            kept.append(cand)

```

B.4 Safety rules (app/agents/safety_agent.py)

```

RULES: Dict[str, List[str]] = {
    "unsupported_certainty": [
        r"\bdefinitely\b", r"\bcertainly\b", r"\bundoubtedly\b", r"\bwithout (a )?
doubt\b",
        r"\bconfirmed diagnosis\b", r"\bdiagnosis is confirmed\b", r"\b100\s?%",
        r"\bguaranteed?\b", r"\bthe patient has\b", r"\bis diagnosed with\b", r"\
bclearly has\b",
        # claiming a step is required (the plan may only say what "could be
considered")
        r"\b(should|must) be (measured|performed|done|obtained|ordered|carried out)\
b",
        r"\b(is|are) (mandatory|essential|warranted|required)\b", r"\bstrongly
supports? the diagnosis\b",
    ],
    "treatment_recommendation": [
        r"\bprescrib\w*", r"\btreatment plan\b",
        r"\bd+(\.\d+)?\s?(mg|mcl|ml|units?|iu)\b(?:!\s*/)", # doses, but not lab
units like mg/L
        r"\b(start|begin|initiate|commence)\w*\s+(on\s+)?(an\s+)?(antibiotics?|
antivirals?|insulin|steroids?|medication|treatment|therapy)\b",
    ]
}

```

```
    r"\b(take|administer|give)\s+(\w+\s+)?(antibiotics?|paracetamol|acetaminophen|
    ibuprofen|insulin|tablets?|medication|dose)\b",
    r"\b(increase|decrease|reduce|stop|discontinue|double)\s+(the\s+|their\s+|his\s+
    |her\s+)?(dose|dosage|medication)\b",
    r"\b(amoxicillin|azithromycin|ceftriaxone|doxycycline|oseltamivir|paracetamol|
    ibuprofen)\b",
  ],
  "unsafe_language": [
    r"\bno need to (see|consult|visit)\b", r"\b(ignore|dismiss)\s+(the\s+|these\s+
    )?symptoms?\b",
    r"\bself[- ]?medicat\w*", r"\bnothing to worry\b",
    r"\b(don't|do not) need (a |to see a )?(doctor|clinician|medical)\b",
    r"\bavoid (seeing|consulting|visiting) (a )?(doctor|clinician|hospital)\b",
  ],
}
```

C. Extra Screenshots

Body systems: constitutional, respiratory

name	drug_class
antihypertensive medication	antihypertensive (class not specified)

5. Laboratory analysis

2 lab value(s) reviewed: 2 outside the reference range, 0 invalid, 0 without a reference range.

test	value	unit	reference_range	status	note
WBC	13500	cells/uL	4000-11000 cells/uL	high	High relative to the educational reference range.
CRP	48	mg/L	0-10 mg/L	high	High relative to the educational reference range.

Reference-range comparison only. An out-of-range value is an observation that clinicians interpret in context; it is not a diagnosis.

6. Retrieved educational context (RAG)

Query: fever cough fatigue hypertension WBC high CRP high

- > KB-001 - Community-acquired pneumonia (score 0.5518)
- > KB-012 - Interpreting inflammatory markers (WBC and CRP) (score 0.5371)
- > KB-005 - Urinary tract infection (score 0.3794)
- > KB-003 - Influenza-like illness (score 0.1699)

7. Differential diagnosis candidates (conditions to consider)

3 condition(s) to consider were derived by matching the case's symptoms, history and out-of-range labs against the retrieved educational knowledge. Levels are relative, not probabilities.

Figure 5: SIM-001: retrieved educational context (RAG) and differential diagnosis candidates

Educational simulation only. This system does not provide medical diagnosis, treatment, or professional medical advice.

Educational Case Report - SIM-001

Patient: 45-year-old male (synthetic case)
Reasoning mode: rule-based
Safety risk level: low

Findings

- Symptoms: The patient reports 3 symptom(s): fever, cough, fatigue. The reported symptom duration is 5 days.
- Warning symptoms: none identified
- History: 1 past condition(s), 1 medication(s) and 0 allergy record(s) reviewed.
- Labs: 2 lab value(s) reviewed: 2 outside the reference range, 0 invalid, 0 without a reference range.
- Out-of-range labs (reference comparison only): WBC high (13500 cells/uL), CRP high (48 mg/L)

Conditions to consider (not a diagnosis)

Figure 6: SIM-001: final educational report tab

Agent outputs 10. Final educational report Agent trace

Messages passed between agents through the shared LangGraph state, in execution order.

agent	status	message	reads
Patient Data Agent	ok	Patient record valid; structured case shared with analysis agents.	patient_input
Medical History Agent	ok	1 past condition(s), 1 medication(s) and 0 allergy record(s) reviewed.	patient
Laboratory Analysis Agent	ok	2 lab value(s) reviewed: 2 outside the reference range, 0 invalid, 0 without a reference	patient
Symptom Analysis Agent	ok	3 symptom(s), systems: constitutional, respiratory.	patient
RAG Retriever	ok	Retrieved 4 document(s): KB-001, KB-012, KB-005, KB-003	symptom_analysis history_analysis
Differential Diagnosis Agent	ok	3 candidate(s) via rule-based reasoning.	symptom_analysis history_analysis
Diagnostic Planning Agent	ok	6 diagnostic step(s) via rule-based reasoning.	symptom_analysis history_analysis
Safety Review Agent	ok	Risk low; passed.	differential_diagnosis diagnostic_pla
Report Generator	ok	Report status: completed.	*

Figure 7: SIM-001: agent-communication trace

Missing / conflicting: HbA1c not measured in this case; Already documented in the history - context rather than a new explanation.

Knowledge refs: KB-007

8. Suggested diagnostic next steps

priority	step	rationale
prompt c	Prompt in-person clinician review of the warning symptoms	The case includes warning features (chest pain, shortness of breath) that,
early	An ECG is typically obtained promptly for chest pain.	Educational reference material lists this when evaluating acute coronary s
early	Cardiac troponin testing is commonly used.	Educational reference material lists this when evaluating acute coronary s
early	Urgent in-person clinical evaluation is expected for suspected cardiac chest pain.	Educational reference material lists this when evaluating acute coronary s
routine	Chest X-ray is commonly used to look for lung consolidation.	Educational reference material lists this when evaluating community-acq
routine	Pulse oximetry is used to assess oxygenation.	Educational reference material lists this when evaluating community-acq
routine	Sputum or blood cultures may be considered in some settings.	Educational reference material lists this when evaluating community-acq
routine	Repeat fasting glucose or HbA1c is commonly used to confirm raised values.	Educational reference material lists this when evaluating type 2 diabetes

9. Safety review

Risk level: high | Passed: True | Human review required: True | Revisions: 0

- Case contains warning symptoms: chest pain, shortness of breath.

Figure 8: SIM-005: diagnostic steps with clinician review first; safety review shows high risk and human review



MedAgent-CDSS

A Multi-Agent Clinical Decision Support System for Educational Simulation

⚠️ Educational simulation only. This system does not provide medical diagnosis, treatment, or professional medical advice. All patient cases are synthetic.

Agent outputs 10. Final educational report Agent trace

1. Patient information

{ ... }

2. Validation result

Patient input is invalid - the workflow stopped before analysis.

- age: Input should be less than or equal to 120
- sex: Field required

Figure 9: SIM-008: invalid input stops the workflow at the Patient Data Agent